SWA Extension — Workflow Specification

Oct 20, 2026 · @Abhishek Shakya

Purpose and scope

The SWA (Software Architect Assistant) extension turns pi into an architect's co-pilot that converts requirements into a reviewed, versioned architecture: a technology stack, a high-level design (HLD), component-level designs (LLD), and conformance reviews of the code that follows.

Every output is a plain-text artifact in Git (Markdown, `.drawio`, `.excalidraw`, YAML), so humans and the agent edit the same files. The architect approves each stage before the next one starts; the assistant proposes, the architect decides.

In scope: requirements intake, stack selection, HLD, design review, LLD, ADRs, code conformance review, change impact analysis.

Out of scope (v1): writing production code, provisioning infrastructure, cost billing integration, enterprise portfolio management.

Inputs

Four inputs feed the workflow; only the SRS and goal.md are mandatory. Inputs are copied into `arch/inputs/` and never edited by the agent, so every later artifact can trace back to a fixed source.

Input	Required	Format	What the assistant takes from it
SRS	Yes	.md, .docx, .pdf	Functional requirements, NFRs, constraints, actors, integrations
goal.md	Yes	.md (template provided)	Business goals, success metrics, timeline, budget, team skills, priorities

Input	Required	Format	What the assistant takes from it
Customer meeting summary	No	.md, .txt	Unwritten expectations, decisions already made, open concerns
Technical governance rules	Yes for enterprise, else default pack	.md or YAML	Approved/forbidden tech, security and compliance rules, naming and coding standards

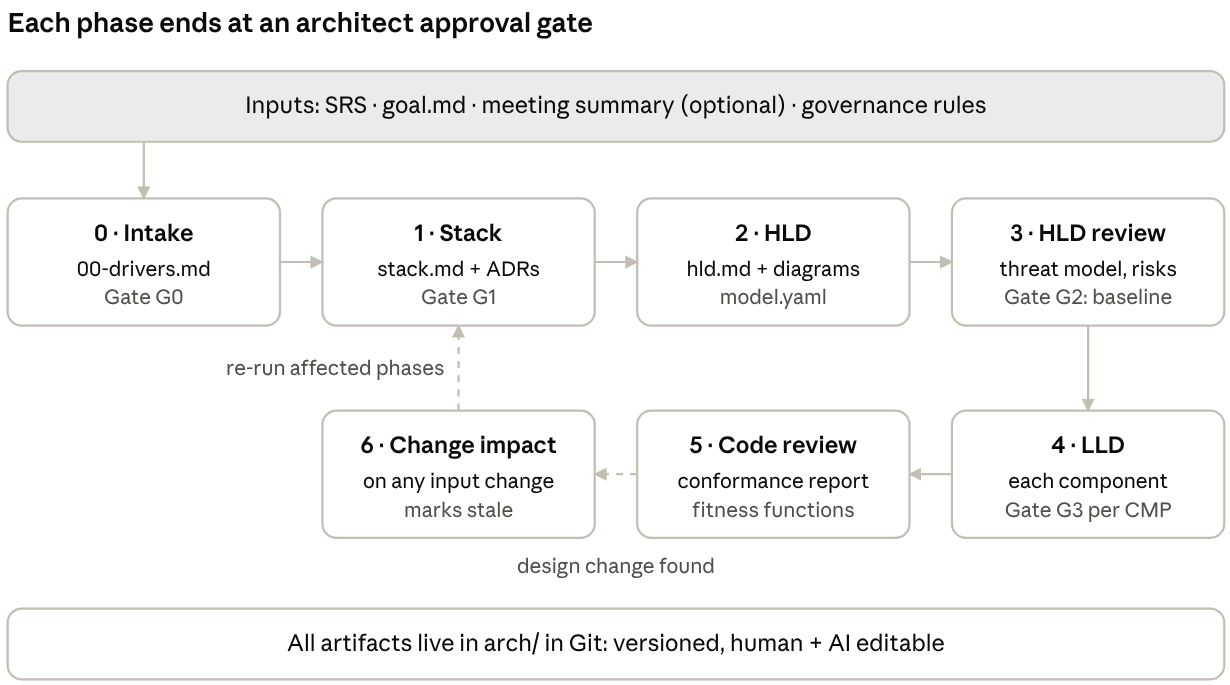
goal.md was undefined in the original workflow, so this spec fixes its template. It answers what the SRS usually leaves out:

- Business goal and the metric that proves it (e.g. 50k MAU in 12 months)
- Delivery deadline and release phases (MVP, v1, v2)
- Budget ceiling for cloud and licences per month
- Team size and current skills (languages, clouds the team already runs)
- Ranked quality priorities (e.g. security > availability > cost > performance)
- Hard constraints (region, data residency, existing systems to integrate)

Governance rules should be machine-readable YAML where possible (`allowed`, `forbidden`, `required` lists), so a tool can check compliance instead of the model guessing.

Workflow overview

The workflow runs seven phases; your original four (stack, HLD, LLD, code review) are kept, and three are added: intake, HLD review and change impact.



SWA workflow · 7 phases, 4 gates, 1 change loop

Solid arrows are the normal path; dashed arrows are the change loop. Nothing moves forward until the architect approves the gate, and LLD runs once per component on request.

Phase specification

The extension defines only the flow: what each phase consumes, what it must produce, and when the architect approves. The agent plans the steps inside a phase itself, using the project context (inputs, approved artifacts, existing ADRs, the repo).

Phase	Command	Reads	Must produce	Gate
0 · Intake	`/swa intake`	SRS, goal.md, meeting summary, governance	`00-drivers.md`: requirement IDs, measurable quality scenarios, conflicts, answered questions (incl. target cloud)	G0

Phase	Command	Reads	Must produce	Gate
1 · Stack	`/swa stack`	Drivers, goal.md, governance	`stack.md` (architecture style, tech per layer, NFR balance, standards) + one ADR per significant choice	G1
2 · HLD	`/swa hld`	Drivers, goal.md, stack.md	`model.yaml`, `hld.md`, context + container + key-flow + deployment diagrams, shown in the app	—
3 · HLD review	`/swa review-design`	HLD, drivers, stack.md	`hld-review.md` (traceability, trade-offs, threat model, cost), `risks.md`	G2 (baseline tag)
4 · LLD	`/swa lld <CMP-id>`	Approved HLD, model.yaml	Per component: `lld.md`, component + sequence diagrams, data model, API/event contracts	G3 per component
5 · Code review	`/swa review-code`	Code, model.yaml, stack.md, contracts	Conformance report (blocker/major/minor, each citing a rule or ADR) + fitness-function test files	—
6 · Change impact	`/swa impact`	Changed input or artifact	Impact report; affected artifacts marked `stale`	Re-enters the earliest affected gate

Rules the agent always follows, whatever plan it makes:

- Never start a phase whose input artifacts are not `approved` unless the architect says so.
- Cite requirement, component and ADR IDs for every design claim.
- Ask the architect when an input is missing or conflicting; never invent requirements.
- Show every diagram in the app and save it as an editable file in `arch/`.
- Never edit text inside `<!-- swa:locked -->` blocks.

Artifact layout and conventions

All artifacts live in an `arch/` folder in the project repo, so Git gives history, diffs, review and rollback for free.

```
arch/
  inputs/            srs.md, goal.md, meeting-summary.md, governance.yaml
  00-drivers.md      requirement IDs + quality scenarios
  stack.md
  model.yaml         component model (source of truth for diagrams)
  decisions/         ADR-0001-architecture-style.md, ...
  hld/               hld.md, context.drawio, containers.drawio,
flows/*.drawio
  lld/CMP-003/       lld.md, component.drawio, sequence-*.drawio,
openapi.yaml
  reviews/           hld-review.md, code-2026-10-20.md, impact-*.md
  risks.md           risk register + tech debt
  .swa/state.json    phase status, used by the extension
```

Human and AI editable means four rules:

- **YAML frontmatter on every Markdown artifact:** `id`, `status` (draft, in-review, approved, stale, superseded), `version`, `owner`, `inputs` (files + hashes it was built from).
- **Stable section headings and IDs** so the agent can update one section without rewriting the file; humans may edit any prose freely.
- **Protected blocks:** text between `<!-- swa:locked -->` markers is architect-owned and the agent must never change it.
- **Diagrams are generated from model.yaml.** Manual edits in draw.io are allowed; `swa_diagram_sync` reads the edited file back and reports differences against the model instead of overwriting them.

Traceability IDs tie everything together: `FR-*` / `NFR-*` → `CMP-*` → `ADR-*` → code paths. These links power the HLD review, code review and impact analysis.

Pi package composition

The extension ships as one pi package (`pi install npm:pi-swa`) that bundles six kinds of parts. Pi loads skills on demand, turns prompt templates into slash commands, accepts TypeScript extensions via `ExtensionAPI`, and lets a project replace or append the system

prompt with `SYSTEM.md` (pi deep dive).

Part	Pi mechanism	Contents
System prompt	`SYSTEM.md` (append)	Architect persona, flow and gate rules, "cite IDs", "show and save every diagram", locked-block rule
Commands	Prompt templates in `prompts/`	`/swa` router plus `intake`, `stack`, `hld`, `review-design`, `lld`, `review-code`, `impact`, `adr`, `export`, `status`
Skills	`SKILL.md` folders	swa-intake, swa-stack, swa-hld, swa-design-review, swa-threat-model, swa-lld, swa-adr, swa-code-review, swa-impact, swa-export, drawio-authoring, excalidraw-authoring, c4-modelling
Custom tools	TypeScript extension	`swa_state` (phase + gates), `swa_validate` (frontmatter/schema), `swa_governance_check`, `swa_trace` (requirement ↔ component matrix), `swa_diagram_save` / `swa_diagram_sync`, `swa_ingest` (docx/pdf → md), `swa_export` (docx/pdf)
MCP servers	Native pi MCP	draw.io (MCP App server), Excalidraw (official MCP), Confluence (for export)
Templates & data	Package assets	goal.md, stack.md, hld.md, lld.md, ADR, review reports, `governance.default.yaml`, JSON schemas

MCP is native. Current pi versions support MCP by default, so the package declares its MCP servers directly; no adapter is needed.

Diagrams render inside the app and are saved as files. Because pi is the product UI, the architect must see and edit diagrams in place. The draw.io MCP App server fits this: `create_diagram` renders draw.io XML as an interactive diagram inline with an "Open in draw.io" button, and `search_shapes` finds shapes across AWS, Azure, GCP and Kubernetes libraries; it can be self-hosted with Docker to keep diagram data in your network. The official Excalidraw MCP renders hand-drawn diagrams inline with fullscreen

editing. Two requirements follow: your pi UI must host MCP App views (the inline iframe), and the agent saves the same XML or Excalidraw JSON to `arch/` so the file in Git and the view in the app always match.

Both tools ship in v1, each with a job: Excalidraw for whiteboard sketches during intake and early HLD discussion; draw.io for every approved HLD and LLD diagram (cloud shape libraries, PNG/SVG/PDF export for `/swa export`).

Gaps found and how they are filled

The original workflow had the right spine (stack → HLD → LLD → code review) but skipped the steps real architects use to make decisions defensible and keep them true over time. Twelve gaps, ranked by impact:

#	Gap in original workflow	Why it matters	Fill
1	No requirement analysis before stack selection	Vague NFRs ("must be scalable") produce arbitrary stacks	Phase 0: IDs + measurable quality scenarios
2	stack.md records *what*, not *why*	Six months later nobody knows why Kafka was chosen	One ADR per significant decision
3	No approval gates	AI output flows into the next phase unreviewed	Status in frontmatter + `swa_state` blocks the next phase until approved
4	No HLD review	Design risks surface during coding, when they cost most	Phase 3: traceability, ATAM-lite, STRIDE, Well-Architected, cost
5	Diagram and doc can drift apart	The picture says one thing, hld.md another	model.yaml as single source; `swa_diagram_sync`
6	LLD is diagrams only	Developers need contracts and data models, not pictures	LLD adds OpenAPI/AsyncAPI, ERD, error handling, observability
7	Code review undefined	Becomes a generic linter	Phase 5: conformance against model, stack, contracts; fitness functions in CI

#	Gap in original workflow	Why it matters	Fill
8	No change handling	An SRS update silently invalidates the design	Phase 6: impact analysis, `stale` status, superseding ADRs
9	goal.md and governance format undefined	Every run interprets them differently	Fixed templates; governance as YAML checked by a tool
10	No architecture style decision step	Style is the most expensive decision to reverse	First step of Phase 1, its own ADR
11	No risk / tech-debt register	Known risks are lost between phases	`arch/risks.md`, fed by every review
12	Diagrams only as files	In an app UI, the architect must see and edit diagrams in place	MCP App inline rendering + the same diagram saved to arch/

Smaller additions: input documents stored read-only for reproducibility; `/swa status` to show where the project stands; consistency check between neighbouring LLDs; locked blocks so the agent never overwrites the architect's own wording.

Decisions for v1

All six open questions are answered; they narrow v1 to one architect, a cloud-agnostic stack, and in-app review with both diagram tools.

#	Question	Decision	Effect on the spec
1	One architect or a team?	One architect	The same user authors and approves; each gate is one "Approve" action in the app
2	Which clouds in v1?	Keep it simple; ask the architect	Stack skill stays cloud-agnostic; intake asks for the target cloud when goal.md doesn't state it

#	Question	Decision	Effect on the spec
3	Company governance or default?	Default pack	v1 ships `governance.default.yaml`; a company file can override it later
4	Where does code review run?	Inside the app only	No CI job or PR bot in v1; fitness-function tests are generated as files for later CI use
5	Excalidraw in v1?	Both draw.io and Excalidraw	Both MCP servers ship in v1 and render diagrams inside the app
6	Export for stakeholders?	Yes	`/swa export` produces Word/PDF and Confluence pages from approved artifacts